



# CFS-Treffen

**25. September 2007**

# Themen

- ❑ Cmake Einführung (Simon)
- ❑ Aufräumarbeiten im Code
- ❑ OLAS (SBM System, CFS Integration)
- ❑ Vorstellung CFSTool / Verwendung in TestSuite
- ❑ Neues Konzept für Funktionenräume in CFS
- ❑ Verwendung von Enum-Klasse
- ❑ Neues Konzept für WriteInfo-Klasse (Fabian)

# Aufräumarbeiten

- ❑ Deaktivierung des Funktionentracings, da kaum verwendet und flexiblere Alternativen vorhanden
  - Logging Mechanismus
  - Callgrind
- ❑ Wird `USE_INTEGLIB` noch gebraucht?
- ❑ `Error()` endgültig durch `EXCEPTION()`-Makro ersetzen
- ❑ Verwendung von `boost::programOptions` statt umständlichen `CommandLineHandler` (Vorteil: kann `environment` parsen -> `XMLSchema`)
- ❑ Globale Daten soweit wie möglich vermeiden v.a. streams (`std::stringstream * warning, *error` )
- ❑ `ParamNode *` auf `shared_ptr<ParamNode>` umsetzen

# Typdefinitionen

- ❑ Macht Definition von Integer, Double und Uint Sinn?
- ❑ Pro:
  - definierte Größe über Verwendung von `<stdint.h>`
  - teilweise kürzer als Standard-Typen (Uint  $\leftrightarrow$  unsigned int)
- ❑ Contra:
  - wird unterschiedliche Genauigkeit jemals benötigt?
  - Syntaxhighlighting funktioniert nicht richtig?

# std::vector<> vs. StdVector<>

- ❑ Ziel: Einheitliche Verwendung einer von beiden Klassen
- ❑ Pro std::vector<>
  - standardisiertes Interface zu externen Klassen
  - etabliert
  - Verwendung von Algorithmen der stl
- ❑ Contra std::vector<>
  - keine automatische Indexprüfung im DEBUG-Fall möglich (nut über at()-Methode)
  - manche Operationen teilweise umständlich zu benutzen (z.B. std::find() )

# std::vector<> vs. StdVector<> (Cont.)

## □ Pro StdVector<>

- Erweiterbarkeit, da selbst implementiert
- Automatische Indexprüfung durch IFDEF
- Als Container für nacktes C-Array verwendbar
- Boost Framework zur Implementierung von standardkonformen Iteratoren implementierbar

## □ Contra StdVector<>

- Beim Interfacing zu externen Bibliotheken Konvertierung notwendig
- Andere Schreibweise als stl-Klassen

# OLAS

## ❑ Probleme:

- extra Parameterhandling über OLASParams-Klasse
  - ⇒ besser: ParamNode mitverwenden
- eigener Namespace OLAS::
  - ⇒ doppelte Klassennamen (z.B. StdVector) verhindern einfache Integration in CFS
- 1-basierte Implementierung der Speicherverwaltung verhindert Nutzung der Klassen in CFS

## ❑ Wichtige Erweiterungen:

- Implementierung des SBM-Systems
- Zuordnung von Matrixblöcken anhand einer FunctionId (bisher: nur PDEId möglich)

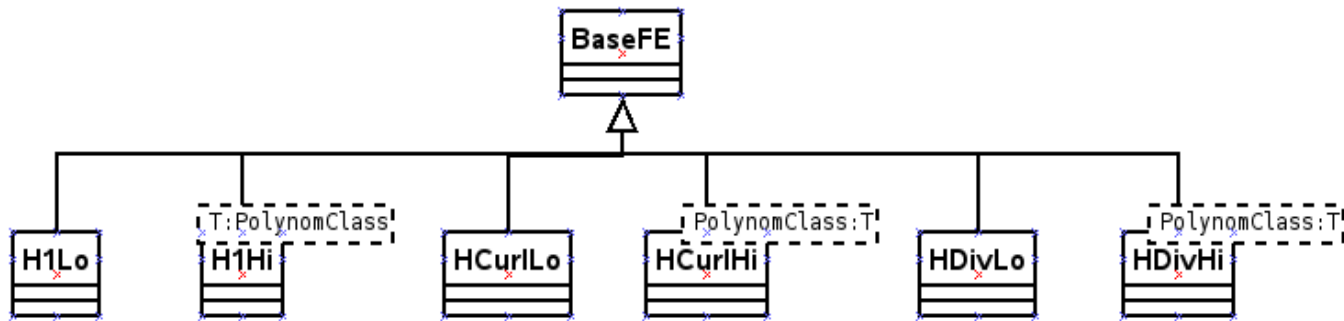
## ❑ Lösung?

# Klassenframework für Functionspace

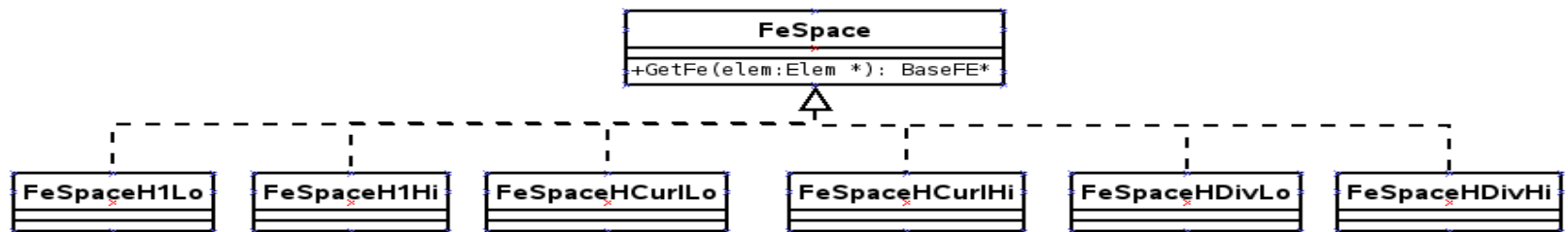
- ❑ **Nachteile der bestehenden Implementierung:**
  - **Referenzelement (BaseFE) fest „verdrahtet“ mit geometrischer Repräsentierung (Elem)**
    - ⇒ schwierige Implementierung verschiedener Räume  $H(\text{div})$ ,  $H(\text{curl})$
  - **Nur isoparametrisches Mapping von Elementen möglich**
  - **Definition und Berechnung von Elementmatrizen braucht viel Code und macht Berechnungen oft doppelt (z.B.  $J^{-1}$  Berechnung)**
  - **Axensymmetrie explizit in jedem Integrator implementiert**
  - **Integrationsregeln nur global definiert, anstatt pro Integrator**
  - **Doppelte Implementierung von B-Operatoren**

# Neue Klassen

- **BaseFE:** Wird nicht mehr primär nach Elementtype untergliedert sonder nach Funktionenraum

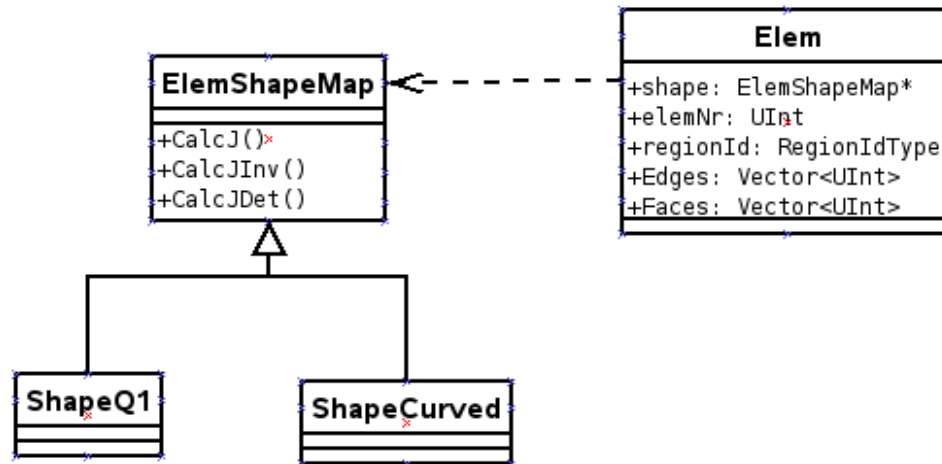


- **FunctionSpace:** Enthält Pointer auf Referenzelemente und gibt für ein Elem\* das BaseFE zurück



# Neue Klassen II

- Trennung von Basiseigenschaften des Elements und der gemapten Repräsentierung

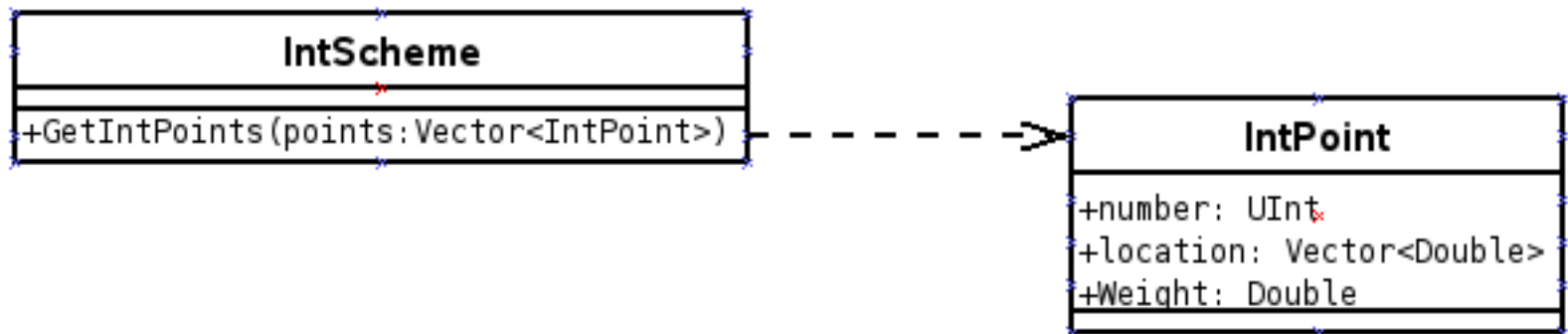


- Vorteil:

- Beliebige, gemischte Elementordnungen pro Gitter möglich
- Kapselung des Attributes „isAxi“
- Jacobi-Matrix wird nur einmal berechnet (z.B. beim ADB-Integrator, wo momentan `CalcAMat()` und `CalcBMat()` J doppelt berechnen und `CalcElementMatrix()`)

# Neue Klassen III

- ❑ Basisfunktionen haben im Moment zwei Interfaces:
  - Calc...Atlp() (nur für Integrationspunkt)
  - Calc...() (für bel. lokale Koordinaten)
- ❑ Nachteil: Doppelte Implementierung des B-Operators:
  - im BDB-Fall nur im Integrationspunkt
  - im Postprocessing: nur im Elementmittelpunkt
- ❑ Lösung:



# Neue Klassen IIII

- **Weiterer Vorteil:**
  - Jede Bilinearform erhält eigene Integrationsordnung
  - Berechnungsmethoden können sowohl für Berechnung von Element-Matrizen/Vektoren, als auch für PostProcessing verwendet werden

# Neue Klassen V

## □ Vereinheitlichung der Klassen

- eqnMap
- NodeStoreSol
- bcs

